

Python 语法手册

—— 配 ServerTimeDemo 项目实例

中册

抽象与组织 · 第六章 ~ 第十章

从词法到工程约定，逐条对照真实代码

74 个 Python 文件 · 44,490 行 · 23 张数据表

项目版本 1.9.0

手册版本 1.0

生成日期 2026 年 09 月 21 日

《Python 语法手册》全三册 · 本册为中册

本手册的每一段示例代码都摘自本项目源码，未作伪写

目录

本册是三册中的**中册**，收录 第六章 ~ 第十章。（抽象与组织）

第六章 函数	4
6.1 定义、返回值、多返回值	4
6.2 参数的四种形态	4
6.3 可变默认参数的陷阱（第 4.7 节的正脸）	5
6.4 作用域：LEGB 与闭包	5
6.5 lambda：只配做「一次性小函数」	7
6.6 函数是对象：回调与高阶函数	8
6.7 装饰器	9
第七章 类与对象	11
7.1 class 与 self	11
7.2 实例属性 vs 类属性	12
7.3 继承与 super()	12
7.4 property：把方法当属性读	13
7.5 classmethod 与 staticmethod	13
7.6 魔术方法：让自定义对象融入语言	15
7.7 dataclass：数据类的标准做法	16
7.8 组合优于继承：本项目的分层就靠它	17
第八章 异常处理	18
8.1 try 语句的四个部分	18
8.2 常见内置异常与层级	18
8.3 raise 与异常链 from	19
8.4 两种处理风格，以及怎么选	20
8.5 assert：只用来钉「不可能发生」	21
8.6 自定义异常	21
第九章 模块、包与导入	23
9.1 导入的四种写法	23
9.2 if __name__ == "__main__"	24
9.3 包、相对导入与 __init__.py	24

9.4 延迟导入：把 800 毫秒挪出启动路径	25
9.5 冻结（打包）后的路径	26
9.6 sys.path 与脚本运行	26
第十章 类型注解	27
10.1 注解写在三个地方	27
10.2 Optional 与联合类型	27
10.3 容器泛型	28
10.4 Callable：把函数当参数传时怎么标注	28
10.5 from __future__ import annotations	29
10.6 注解不参与运行 —— 一条必须记住的边界	30

第六章 函数

函数是 Python 里唯一的抽象单位：它同时是「一段逻辑」和「一个对象」。GUI 编程之所以处处是函数，就是因为按钮、绑定、定时器全都要求「传一个可调用对象进来」。

6.1 定义、返回值、多返回值

没有 `return` 的函数返回 `None`。要返回多个值时，**返回的是元组**，调用方解包即可 —— 不需要任何特殊语法：

```
1 def _resolve_level(level):
2     """确定日志级别：显式参数 > 环境变量 > 默认 INFO。"""
3     if level is not None:
4         return level
5     raw = os.environ.get(ENV_LEVEL, "").strip().upper()
6     if raw:
7         value = getattr(logging, raw, None)
8         if isinstance(value, int):
9             return value
10    return DEFAULT_LEVEL
```

▲ `log_setup.py` · 三级优先级的兜底写法

注意这个函数的返回值**可能是调用方传进来的同一个对象**（`return level`）。这是有意的：不加工就直接原样传回去，把「判断」和「构造」分开。

6.2 参数的四种形态

形态	写法	说明
位置参数	<code>def f(a, b):</code>	按顺序传
默认参数	<code>def f(a, b=0):</code>	必须在位置参数之后
可变位置	<code>def f(*args):</code>	<code>args</code> 是元组
可变关键字	<code>def f(**kwargs):</code>	<code>kwargs</code> 是字典
仅关键字	<code>def f(a, *, b):</code>	<code>*</code> 之后的只能用 <code>b=</code> 传
仅位置	<code>def f(a, /, b):</code>	<code>/</code> 之前的不能按关键字传 (3.8+)

仅关键字参数在本项目里用得很刻意。看这个界面组件的构造函数：

```
1 def __init__(self, master, *, bg: str, inner_bg: Optional[str] = None):
2     super().__init__(master, bg=bg)
3     self._bg = bg
```

▲ `todo_page.py` · `ScrollArea.__init__` 的 `*`

* 是在防止什么

`bg` 与 `inner_bg` 都是颜色字符串，位置传参时写反了是一个安静的 bug —— 界面底色和内容底色对调，程序照样跑，只是看起来怪。加上 `*` 之后，调用处必须写 `ScrollArea(parent, bg="#fff", inner_bg="#fafafa")`，两个颜色各自带着名字，顺序就再也不会搞错。

同理 `PendingAlerts` 那种「一次调三个开关」的函数更适合全部改成仅关键字。

6.3 可变默认参数的陷阱（第 4.7 节的正脸）

```
1 def add(item, bucket=[]):           # 危险：默认值只在定义时求值一次
2     bucket.append(item)
3     return bucket
4
5
6 add(1)                               # [1]
7 add(2)                               # [1, 2]  <- 上一次的 1 还在
```

▲ 反例（由本手册编写）

根因是「默认值只在定义时求值一次」

函数（和方法）的默认值在定义时求值一次，之后所有调用共用同一个对象。

所以上面那个空列表会被所有调用共享 —— 往里 `append` 一次，下次调用它还在。3.7 节讲的 `field(default_factory=list)` 是同一个坑的 `dataclass` 版本。

规矩：默认值一律用不可变对象；可变的一律用 `None` 哨兵或工厂函数。

```
1 def build_tree(nodes, seen=None):
2     seen = seen if seen is not None else set()   # 每次调用现造一个
3     ...
```

▲ 用 `None` 当哨兵的标准写法

6.4 作用域：LEGB 与闭包

Python 找名字的顺序是 **Local** → **Enclosing**（外层函数）→ **Global**（模块）→ **Builtin**（内建）。函数内部只能读外层的变量；要写，必须声明 `global`（写模块级）或 `nonlocal`（写外层函数级）。

```

1  _STRIKE_FONTS: dict = {}
2
3
4  def strike_font(base, widget):
5      """把元组字体转成带删除线的 ``tkinter.font.Font``。"""
6      key = tuple(base)
7      cached = _STRIKE_FONTS.get(key)      # 读全局字典：允许
8      if cached is not None:
9          try:
10             cached.actual()
11             return cached
12         except tk.TclError:
13             _STRIKE_FONTS.pop(key, None)  # 改全局字典的内容：也允许
14             font = tkfont.Font(root=widget, family=base[0], size=base[1],
15                               weight=("bold" if "bold" in styles else "normal"),
16                               overstrike=1)
17             _STRIKE_FONTS[key] = font      # 仍然是改内容，不是重新赋值
18     return font

```

▲ todo_page.py · 模块级缓存 + 函数读写标准姿势



▲ 完成态的两种标记同时生效：左侧勾选 + 标题文字中间一道删除线。这道横线只有 `overstrike=1` 的 `Font` 对象画得出来——上面那个字体缓存就是为它服务的。

「改字典内容」为什么不需要 `nonlocal` / `global`

规则是：**给名字重新赋值**才需要声明；`d[k] = v`、`d.pop(k)`、`lst.append(x)` 都不是重新赋值，它们只是在调用对象自己的方法。所以上面这段完全合法。如果写成 `_STRIKE_FONTS = {}`（重新绑定这个名字），函数里就必须先声明 `global`，否则 Python 会在函数内新建一个同名局部变量，外面那个纹丝不动——**这是最安静的一类 bug**。

闭包：函数记住了它出生时的环境

```

1  def make_strike_fontter(base):
2      """返回一个「把 base 变成带删除线字体」的函数。"""
3      cache = {}
4
5      def get(widget):
6          font = cache.get(widget)
7          if font is None:
8              font = tkfont.Font(root=widget, family=base[0], size=base[1],
9                               overstrike=1)
10             cache[widget] = font
11             return font
12
13     return get          # get 记住了 base 和 cache

```

▲ 闭包演示（由本手册编写，用于说明内层函数如何捕获外层变量）

```

1  for i in range(3):
2      buttons.append(lambda: print(i))    # 三个都会打印 2，不是 0 1 2

```

▲ 反例（由本手册编写）

闭包捕获的是「变量」而不是「当时的值」

三个 lambda 捕获的是**同一个变量 i**，循环结束时 i 已经是 2，所以点哪个按钮都打印 2。

两种修法：① 用默认参数把当前值**绑死在定义时**，写 `lambda i=i: print(i)`；② 抽出一个工厂函数，让 i 成为那次调用的局部变量。

这个坑在 GUI 里是高频雷区，因为「循环创建一排按钮」正是最常见的写法。

```

1      for node in nodes:
2          self._make_row(node, level=level, kind=kind)
3
4      def _make_row(self, node, *, level, kind):
5          """每行都由这个方法独立创建，node 是本次调用的局部变量。"""
6          row = tk.Frame(self)
7          row.bind("<Button-1>", lambda _e, n=node: self._on_click(n))

```

▲ nav_sidebar.py 的做法：用工厂方法（而非在循环里直接写 lambda）

上例中 `lambda _e, n=node: ...` 用了「默认参数绑定」这一招：`n=node` 在**定义时**就把当前的 `node` 存进了函数的默认值里，之后再怎么循环都与它无关。

6.5 lambda：只配做「一次性小函数」

lambda 的语法是 `lambda 参数: 表达式`，函数体必须是一个表达式，不能有语句、不能有 return、不能多行。它存在的意义只有一个：给 `key=`、`command=`、`bind()` 这类要「一个小函数」的地方当场塞一个。

```
1         self.bind("<Enter>", lambda e: self.set_hover(True))
2         self.bind("<Leave>", lambda e: self.set_hover(False))
3         if command is not None:
4             self.bind("<Button-1>", self._invoke)
```

▲ ui_components.py · 悬停进出用 lambda，真正的点击回调用具名方法

注意这段的分界很讲究：**纯转发一行调用的用 lambda，有实际逻辑的用方法**。lambda e: self.set_hover(True) 一眼就看完，而 _invoke 里要做状态判断、要触发回调、要返回值，所以它有名字。

场景	选择
key=lambda x: x[0]	lambda
command=lambda: self.do(1)	lambda
需要 try / 多行 / 分支	写具名函数，再传函数名
会被反复调用、需要单元测试	写具名函数
需要 __name__ 出现在日志里	写具名函数（lambda 的名字是 <lambda>）

6.6 函数是对象：回调与高阶函数

这是理解 Tkinter 的关键：**command=on_click 传的是函数对象本身，command=on_click() 传的是它的返回值**——后者会在绑定时立刻调用一次，然后把返回值（通常是 None）当作回调。

```
1         dialog = TodoAlertDialog(
2             self, items, db=self.todo_db,
3             on_changed=self._after_todo_alert,      # 传函数对象，不加括号
4             on_open=self._open_todo_item,
5         )
```

▲ main.py · show_todo_alert() · 三个回调全是具名方法

常见的更高阶用法还有 functools.partial 与把函数放进容器里派发：

```
1  from functools import partial
2
3  # partial: 把部分参数先固定住，得到一个新的可调用对象
4  on_pick = partial(self.set_color, picker, hex_value)
5  on_pick()          # 等价于 self.set_color(picker, hex_value)
6
7  # 派发字典：按 key 找处理函数，比长串 if / elif 清楚
8  HANDLERS = {
9      "module_ops_expiry": self._build_expiry_page,
10     "module_life_todo": self._build_todo_page,
11 }
12 HANDLERS[key]()
```

▲ 由本手册编写；项目中等价的写法见 main.py 的页面切换

`__name__` 与 `functools.wraps`

函数对象自带 `__name__`、`__doc__`、`__module__`。写装饰器时如果不把这几个属性搬过去，被装饰的函数在日志里就会显示成包装函数的名字——本项目的 `logger.exception` 会带上模块名，所以装饰器一定要用 `functools.wraps`。见 6.7。

6.7 装饰器

装饰器就是「接收一个函数、返回一个新函数」的函数。`@deco` 写在 `def` 上方，等价于在这句 `def` 之后执行 `func = deco(func)`。

项目里在用的四种内建装饰器

装饰器	作用	项目里的位置
<code>@property</code>	把方法伪装成属性，读的时候不加括号	<code>TodoItem.due_day</code> 、 <code>ReminderSummary.has_items</code>
<code>@classmethod</code>	第一个参数是类本身（ <code>cls</code> ），常当「备用构造函数」	<code>TodoItem.from_row()</code> 、 <code>ToolboxDatabaseAdapter.from_source()</code>
<code>@staticmethod</code>	不接收 <code>self / cls</code> ，只是挂在类名下的普通函数	<code>todo_page.py</code> 里的三个纯函数
<code>@dataclass</code>	按注解自动生成 <code>__init__ / __repr__ / __eq__</code>	<code>TodoItem</code> 、 <code>TodoList</code> 、 <code>Typography</code> 、 <code>Palette</code>

```
1 @dataclass(frozen=True)
2 class Typography:
3     """字体 token（字号单位为 pt，10pt 约等于 13px）。
4
5     注意：Tk 只支持 normal / bold 两档字重（没有 500），
6     因此「只用两种字重」在这一层映射为 normal 与 bold。
7     """
8
9     hero: tuple = ("Microsoft YaHei UI", 18)
10    page_title: tuple = ("Microsoft YaHei UI", 16)
11    title: tuple = ("Microsoft YaHei UI", 11, "bold")
12    body: tuple = ("Microsoft YaHei UI", 10)
13    caption: tuple = ("Microsoft YaHei UI", 9)
```

▲ `ui_theme.py` · 用 frozen dataclass 当「常量表」

`frozen=True` 换来了什么

冻结之后实例属性不可再赋值，`Typography()` 本身又不可哈希（字段是 tuple 可以），关键是它向读代码的人宣告了「这是常量」。如果哪天有人写了 `TYPOGRAPHY.body = 12`，会立刻抛 `FrozenInstanceError`，而不是悄悄改坏全局样式。

自己写一个装饰器

```

1 import functools
2 import time
3
4
5 def timed(logger):
6     """记录被装饰函数的耗时。"""
7
8     def deco(func):
9         @functools.wraps(func)      # 保住 __name__ / __doc__
10         def wrapper(*args, **kwargs):
11             start = time.perf_counter()
12             try:
13                 return func(*args, **kwargs)
14             finally:
15                 cost = (time.perf_counter() - start) * 1000
16                 logger.debug("%s 用了 %.1f ms", func.__name__, cost)
17
18         return wrapper
19
20     return deco
21
22
23 @timed(logger)
24 def load_workbook(path):
25     ...

```

▲ 带参数的装饰器：外三层、里一层

`functools.wraps` 不能省

少了它，被装饰的函数 `__name__` 会变成 `wrapper`、`__doc__` 会变成 `None`。后果是：日志里看到的全是 `wrapper`，`help()` 查不到文档，测试里按名字找函数也找不到。**所有手写装饰器都必须带 `functools.wraps`。**

```

1 @a
2 @b
3 def f(): ...
4
5 # 完全等价于
6 f = a(b(f))

```

▲ 多层装饰器

装饰器的执行顺序（从下往上）

离 `def` 最近的那个**先包上**，所以执行顺序是 `b` 先、`a` 后。记忆口诀是「**从上往下读，从下往上执行**」。

第七章 类与对象

本项目的界面全是类：每个页面一个 `class XxxPage(ttk.Frame)`，每个数据层一个 `class XxxDB`，每个独立窗口一个 `class XxxWindow`。这一章把类相关的语法一次讲清，并说明项目为什么这么组织。

7.1 class 与 self

```
1 class TodoDB:
2     """待办模块的数据层。
3
4     用法::
5
6         db = TodoDB(DATA_DIR / "expiry_manager.db")
7         lists = db.fetch_lists()
8         db.close()
9
10    多个页面共享同一个连接，所以**不要**在这里 commit 之后就 close。
11    """
12
13    def __init__(self, db_path: Path):
14        self.db_path = db_path
15        self.conn = sqlite3.connect(str(db_path), check_same_thread=False)
16        self.conn.row_factory = sqlite3.Row
17        self.conn.execute("PRAGMA foreign_keys = ON")
18        self._holidays_cache: Optional[dict[str, Holiday]] = None
19
20        self.create_tables()
21        self.seed_holidays()
22        self.seed_default_lists()
```

▲ todo_db.py · 节选（省略了 docstring 中的示例细节）

`self` 就是「这个实例本身」，必须显式写在参数表第一位。它不是关键字，只是一个约定俗成的名字——但请永远叫它 `self`，改成别的只会让人皱眉。

构造函数里这几行的顺序是有讲究的：**先建连接、先设 `row_factory`、再建表**。因为 `create_tables()` 内部要用 `self.conn`；而 `row_factory` 决定了后续所有查询返回的是 `sqlite3.Row` 还是元组——项目里所有 `row["name"]` 这种按列名取值，全靠它。

参数	含义	项目里的选择
<code>check_same_thread=False</code>	允许连接跨线程使用	定时器与子线程都要读库
<code>row_factory = sqlite3.Row</code>	查询结果可按列名取值	全项目通用写法
<code>PRAGMA foreign_keys = ON</code>	开启外键约束	SQLite 默认是关的， 必须显式开

7.2 实例属性 vs 类属性

写法	位置	存在哪	改动影响
<code>self.x = 1</code>	通常在 <code>__init__</code> 里	每个实例各一份	只影响该实例
<code>x = 1</code>	类体里（方法外）	所有实例共用一份	改类属性会影响所有实例

```

1 class TodoItem:
2     # 类属性：这是「字段的默认值」，dataclass 会据此生成 __init__
3     id: int = 0
4     title: str = ""
5     tags: list[str] = field(default_factory=list)

```

▲ todo_db.py · 带注解的类属性被 @dataclass 收编成实例属性

类属性如果是可变对象，就是第 4.7 节的坑

`tags: list = []` 看着像「每个实例一个空列表」，实际是**所有实例共用一个**。这也是为什么 `field(default_factory=list)` 必须写——dataclass 本身不会帮你把类属性拷贝成实例属性，它只是照抄默认值。

7.3 继承与 super()

本项目的继承几乎全部只有一层，而且**全是继承 GUI 框架的基类**：

```

1 class ScrollArea(tk.Frame):
2     def __init__(self, master, *, bg: str, inner_bg: Optional[str] = None):
3         super().__init__(master, bg=bg)    # 先让父类把控件建好
4         self._bg = bg
5
6         self.vsb = ttk.Scrollbar(self, orient="vertical")
7         self.vsb.pack(side="right", fill="y")
8
9         self.canvas = tk.Canvas(self, bg=bg, highlightthickness=0, bd=0,
10                                yscrollcommand=self.vsb.set)
11         self.canvas.pack(side="left", fill="both", expand=True)
12         self.vsb.configure(command=self.canvas.yview)

```

▲ todo_page.py · ScrollArea

`super().__init__()` 必须在最前面

父类 `tk.Frame.__init__` 负责创建真正的控件对象。在它执行之前，`self` 还不是一个可用的控件——此时调用 `self.pack()` 或传 `self` 当父容器都会报 `TclError`。**规矩：子类 `__init__` 第一句永远是 `super().__init__(...)`。**

多层继承时 `super()` 会按 MRO（方法解析顺序）找下一个类。查看顺序用 `ClassName.__mro__`。本项目没用到菱形继承，所以 `super()` 的效果就是「父类」——**但仍然要用 `super()` 而不是 `tk.Frame.__init__(self, ...)`**，因为将来插入中间类时前者自动适配。

7.4 property：把方法当属性读

```

1  @dataclass
2  class TodoItem:
3      # -- 展示辅助 -----
4      @property
5      def due_day(self) -> Optional[date]:
6          """截止日期的 date 形式；未设置时返回 None。"""
7          return parse_day(self.due_date)
8
9      @property
10     def priority_mark(self) -> str:
11         return PRIORITY_MARKS.get(self.priority, "")
12
13     @property
14     def repeat_label(self) -> str:
15         return REPEAT_LABELS.get(self.repeat_rule, "永不")

```

▲ todo_db.py · TodoItem 的三个计算属性

调用处因此变得很干净：`item.repeat_label` 而不是 `item.repeat_label()`。判断该不该用 **property** 的标准是「它像不像一个属性」——「这条待办重复吗」是属性，「把这条待办推进到下一周期」是动作，后者必须是方法（项目里叫 `advance_repeat` 之类）。

只读 property 与可写 property

```

1  class Node:
2      def __init__(self):
3          self._active = False
4
5      @property
6      def is_active(self) -> bool:          # 读
7          return self._active
8
9      @is_active.setter
10     def is_active(self, value: bool):     # 写（名字必须是 <属性名>.setter）
11         self._active = bool(value)
12         self._paint()

```

▲ 由本手册编写；ui_components.py 里是把读写拆成 `is_active` 属性 + `set_active()` 方法

项目里为什么不给每个属性都配 setter

ui_components.py 里只定义了 `@property is_active`（只读），写用的是具名方法 `set_active(value)`。原因：**setter 会让「赋值」看起来是免费的，而这里赋值要触发重绘**。改成方法后，调用方一眼看得出「这一步有副作用」。凡是「写」会引发别的事情（重绘、落库、发通知）的地方，都不该用 setter。

7.5 classmethod 与 staticmethod

```
1 @dataclass
2 class TodoList:
3     id: int = 0
4     name: str = ""
5     color: str = DEFAULT_LIST_COLOR
6
7     @classmethod
8     def from_row(cls, row: sqlite3.Row) -> "TodoList":
9         return cls(
10             id=int(row["id"]),
11             name=_norm(row["name"]),
12             color=_norm(row["color"]) or DEFAULT_LIST_COLOR,
13             sort_order=int(row["sort_order"] or 0),
14         )
```

▲ todo_db.py · from_row() 是典型的「备用构造函数」

	第一个参数	调用方式	典型用途
普通方法	self	obj.m()	需要访问实例状态
@classmethod	cls	Cls.m() 或 obj.m()	备用构造函数；工厂
@staticmethod	无	Cls.m()	逻辑上属于这个类、但不需要实例

from_row 用 cls 而不是 TodoList 的原因

cls(...) 会在子类调用时自动构造子类实例。如果写死 TodoList(...)，将来继承出一个 TodoItemExt 再调 TodoItemExt.from_row(row)，拿到的却是一个 TodoList —— 这类 bug 非常难找。工厂方法一律用 cls。

```
1 class ToolboxDatabaseAdapter:
2     """系统工具箱数据库适配器，统一收口原始 sqlite 连接的传递方式。"""
3
4     def __init__(self, conn: sqlite3.Connection):
5         self._conn = conn
6
7     @classmethod
8     def from_source(cls, db_source):
9         if isinstance(db_source, cls):
10             return db_source # 已经是适配器，原样返回
11         if hasattr(db_source, "conn"):
12             return cls(db_source.conn) # 传进来的是上层 DB 对象
13         return cls(db_source) # 传进来的是裸连接
14
15     @property
16     def connection(self) -> sqlite3.Connection:
17         return self._conn
```

▲ tools_db.py · 兼容三种入参的适配器工厂

isinstance(db_source, cls) 这一句是幂等的关键：同一个适配器被传进去两次，第二次原样返回，不会套娃。

7.6 魔术方法：让自定义对象融入语言

魔术方法（dunder methods）是「用自定义类去接语言的操作符」的钩子。本项目实际用到的不多，但都用在了点子上：

魔术方法	什么时候被调用	项目里的位置
<code>__init__</code>	构造实例	随处可见
<code>__repr__</code>	<code>repr(obj)</code> 、调试器窗口、打印列表	<code>dataclass</code> 自动生成
<code>__eq__</code>	<code>==</code> 比较	<code>dataclass</code> 自动生成（按字段逐一比）
<code>__enter__</code> / <code>__exit__</code>	<code>with</code> 语句进入 / 退出	数据库连接的上下文管理
<code>__iter__</code>	<code>for x in obj</code>	工作日推算器
<code>__len__</code>	<code>len(obj)</code>	容器类组件
<code>__getitem__</code>	<code>obj[key]</code>	行数据包装

```
1 import sqlite3
2 from contextlib import contextmanager
3
4
5 @contextmanager
6 def connect(db_path):
7     """with 写法：无论正常结束还是抛异常，连接都会被关掉。"""
8     conn = sqlite3.connect(str(db_path))
9     conn.row_factory = sqlite3.Row
10    try:
11        yield conn
12    finally:
13        conn.close()
14
15
16 with connect(DATA_DIR / "expiry_manager.db") as conn:
17     rows = conn.execute("SELECT * FROM accounts").fetchall()
```

▲ 由本手册编写；项目里 todos 用的是「长连接 + 显式 close」，见下方说明

项目为什么没用 with 管连接

因为待办模块的连接是跨页面、跨定时器共享的长期连接（`todo_alert_check` 每 30 秒要用一次，提醒窗里点「完成」也要用）。每次操作都开关一次连接，在 SQLite 上意味着反复做文件级加锁，既慢又容易撞上 `database is locked`。

`with` 适合「用完即弃」的资源（临时文件、一次性查询），不适合长生命周期对象。判断标准是：这个资源的生命周期跟「一次操作」对齐，还是跟「整个程序」对齐。

`__repr__` 为什么值钱

`@dataclass` 会自动生成 `__repr__`，于是打印一条待办长这样：`TodoItem(id=7, title='交房租', due_date='2026-09-25', ...)`。调试时能直接看到字段名，比看 `(7, 3, '交房租', '')` 强太多——这是项目大量使用 `dataclass` 的直接收益之一。

7.7 dataclass：数据类的标准做法

```
1 @dataclass
2 class ReminderSummary:
3     """到期提醒摘要数据类。
4
5     属性：
6     overdue: 已过期资产列表
7     due_15:  15 天内到期资产列表（同结构）
8     due_30:  30 天内到期资产列表（同结构）
9
10    方法：
11    has_items -> bool 返回是否还有需要提醒的资产
12    """
13    overdue: list
14    due_15: list
15    due_30: list
16
17    @property
18    def has_items(self) -> bool:
19        return bool(self.overdue or self.due_15 or self.due_30)
```

▲ main.py · ReminderSummary

参数	作用
<code>@dataclass</code>	按注解生成 <code>__init__</code> / <code>__repr__</code> / <code>__eq__</code>
<code>frozen=True</code>	不可变；赋值抛 <code>FrozenInstanceError</code>
<code>eq=False</code>	不生成 <code>__eq__</code> （想用「同一性」比较时）
<code>order=True</code>	生成 <code><</code> <code>></code> 等比较方法（按字段顺序比）
<code>slots=True</code>	生成 <code>__slots__</code> ，省内存、防手滑加属性（3.10+）
<code>field(default=...)</code>	给字段加额外设置
<code>field(default_factory=list)</code>	可变默认值必须用它
<code>field(compare=False)</code>	参与比较时忽略该字段
<code>field(repr=False)</code>	打印时不显示（密码之类）

给 dataclass 加字段一定要带默认值

项目的 `TodoItem` 有 24 个字段，每个都有默认值。原因是**只要有一个字段没默认值，所有 `TodoItem(...)` 的调用点都必须补上它**——而调用点在数据层与界面层一共有几十处。带默认值之后，新增字段（`alerted_for`、`snooze_until` 就是这么加的）只需要动 `dataclass` 定义本身，其它代码一行都不用改。

代价是**字段顺序变得重要**：位置传参会按声明顺序对号入座。所以项目里所有 `TodoItem(...)` 都用关键字传参。

dataclass 与 NamedTuple、普通的类怎么选

数据为主、没有行为 → `@dataclass`。**不可变、还要能当字典键** → `NamedTuple` 或 `frozen=True` 的 `dataclass`。**有明显的状态与行为** → 普通类（比如 `TodoDB`、`ScrollArea`）。本项目的 `TodoDB` 虽然是「数据层」，但它是**行为的集合**，所以是普通类而不是 `dataclass`。

7.8 组合优于继承：本项目的分层就靠它

本项目只有「页面继承框架基类」这一种继承，其余关系全部是**组合**——对象把别的对象当零件用：

```

1  ExpiryManagerApp（主窗口）
2      |—— self.todo_db      = TodoDB(...)          # 数据
3      |—— self.todo_view    = TodoPage(...)         # 界面
4      |—— self.tray         = TrayController(...)   # 托盘
5      |—— self.todo_view 也持有 self.todo_db（注入）

```

▲ 组合关系（真实字段名以 `main.py` 为准）

组合的好处在这里很具体：`TodoPage` **只认「一个长得像 `TodoDB` 的东西」**，所以测试时塞一个替身进去就能跑界面测试，完全不需要真数据库。如果用继承（`class TodoPage(TodoDB)`），页面和数据就焊死了。

拿不准时的判断标准

问一句「**A 是不是一种 B**」。 `TodoPage` 是一种 `tk.Frame` 吗？是——用继承。 `TodoPage` 是一种 `TodoDB` 吗？不是，它是**用一个 `TodoDB`**——用组合。这条「is-a / has-a」的判断在 90% 的情况下够用。

第八章 异常处理

异常不是错误，是**控制流的一种**。Python 的哲学是「先让它抛出来，在知道怎么处理的地方处理掉」，而不是层层返回错误码。但「在哪里处理」和「处理成什么样」是本项目花了不少笔墨的地方。

8.1 try 语句的四个部分

部分	执行时机	用途
<code>try:</code>	总是	放可能抛异常的代码
<code>except E:</code>	try 里抛出 E 时	处理；可写多个， 从上往下匹配
<code>else:</code>	try 里没抛异常时	放「只在成功后才做」的事，别塞进 try
<code>finally:</code>	无论如何	清理：关文件、恢复状态、续期定时器

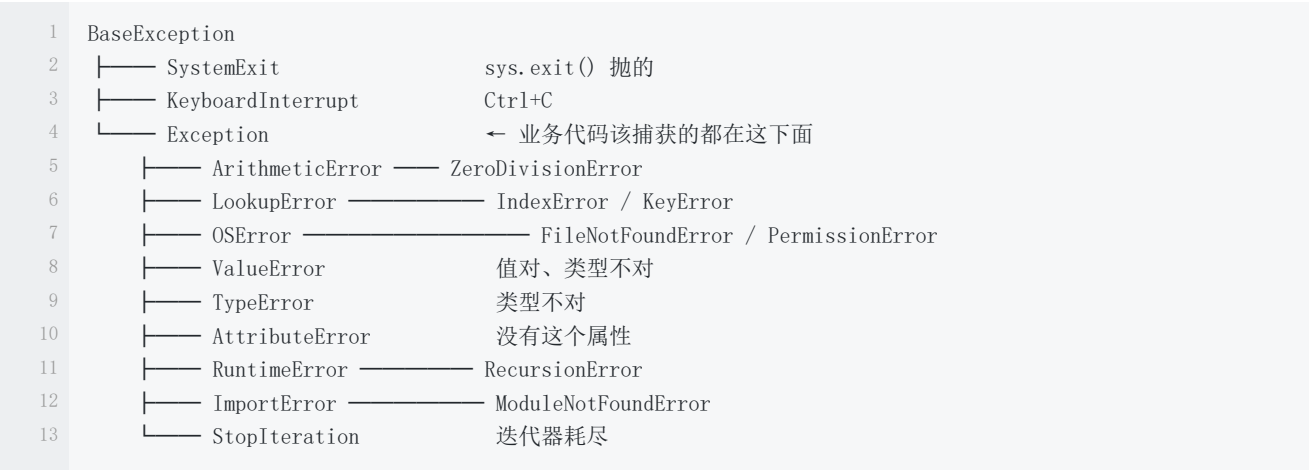
```
1 def todo_alert_check(self):
2     """巡检一次「有没有提醒该响了」。"""
3     if getattr(self, "_exiting", False):
4         return
5     try:
6         self._run_todo_alerts()
7     except Exception:
8         logger.exception("待办到点提醒巡检失败")
9     finally:
10        self.after(TODO_TICK_MS, self.todo_alert_check)
```

▲ main.py · 一个把四个部分用到极致的例子（这里只用了三个）

为什么续期放在 finally 而不是 try 的末尾
这是本项目的定时器纪律：**「续期」必须无条件执行**。
如果写成 `try: 干活(); self.after(...)` —— 一旦「干活」抛异常，`after` 永远不会被调用，**这个定时器就彻底死了**，之后再也不会有任何提醒。这类 bug 用户不会看到报错，只会觉得「提醒时好时坏」。测试里专门塞了一个会抛异常的替身来钉住这条。

`except Exception` **而不是裸** `except:`
裸 `except:` 会连 `KeyboardInterrupt` (Ctrl+C) 和 `SystemExit` 一起吞掉，让程序无法正常中断。项目里所有兜底都是 `except Exception`，只有极少数「清理路径」才用裸 `except` 且立刻继续抛。

8.2 常见内置异常与层级



▲ 只需要记住三件事：业务兜底捕 Exception、LookupError 管下标和键、其余按需

异常	典型触发	本项目里的位置
ValueError	int("abc")、strptime 格式不符	parse_moment() 拆时间
KeyError	d["不存在的键"]	优先用 d.get(k, 默认) 规避
TypeError	1 + "a"、参数个数不对	编码 bug，不该捕获
AttributeError	None.foo、访问不存在的方法	import pygments 之后取属性
OSError / FileNotFoundError	文件不存在、权限不足	日志目录建不出来时降级
sqlite3.Error	SQL 错误、表不存在	数据层
tk.TclError	控件已销毁、字体无效	界面层点击态与字体缓存
ImportError	可选依赖没装	openpyxl 的延迟导入

8.3 raise 与异常链 from

```
1 def load_excel_assets(file_path: Path) -> list[dict]:
2     try:
3         from openpyxl import load_workbook
4     except ImportError as exc:
5         raise RuntimeError("缺少 openpyxl, 请先执行 pip install -r requirements.txt") from exc
```

▲ main.py · 把「依赖没装」翻译成「用户看得懂的一句话」

raise ... from exc 保留了**原始异常**，Python 会把两条都打印出来：

```
1 Traceback (most recent call last):
2   File "main.py", line 2323, in load_excel_assets
3     from openpyxl import load_workbook
4 ModuleNotFoundError: No module named 'openpyxl'
5
6 The above exception was the direct cause of the following exception:
7
8 RuntimeError: 缺少 openpyxl, 请先执行 pip install -r requirements.txt
```

▲ from 会把「直接原因」写在上面，排查时两层都能看到

写法	效果
<code>raise</code>	在 <code>except</code> 里原样重抛，保持原异常
<code>raise NewError(...)</code>	抛新的，原异常挂成 <code>__context__</code> （隐式链）
<code>raise NewError(...) from exc</code>	显式链 ，明确「是它导致的」
<code>raise NewError(...) from None</code>	掐断链 ，只显示新异常（当原异常是噪音时）

什么时候该 `from None`

当原始异常对使用者毫无意义、只会吓人时 —— 比如「配置文件里某个字段格式不对」，底层抛的是 `KeyError: 'port'`，对用户没用。这时候 `from None` 只留自己那句。但**调试期不要掐** —— 排障时你正需要那层原始信息。

8.4 两种处理风格，以及怎么选

本项目里两种风格都在用，而且分得很清楚：**记日志** vs **静默降级**。

风格一：记日志（出问题了我要知道）

```
1 def _after_todo_alert(self):
2     """提醒窗里点过「完成」/「稍后提醒」之后刷新待办页。"""
3     try:
4         self.todo_view.refresh()
5     except Exception:
6         logger.exception("提醒处理后刷新待办页失败")
```

▲ `main.py · logger.exception` 会自动带上完整堆栈

`logger.exception` 等价于 `logger.error(..., exc_info=True)`，只能在 `except` 块里用（它靠 `sys.exc_info()` 取当前异常）。它的输出会落到 `ExpiryManager_Data/logs/app.log`。

风格二：静默降级（失败也无关紧要）

```
1 import pygments
2
3
4 def _get_highlighter():
5     try:
6         from pygments.lexers import PythonLexer
7         from pygments.formatters import HtmlFormatter
8     except ImportError:
9         return None          # 没装 pygments 就没有语法高亮，功能照常
10    return PythonLexer(), HtmlFormatter()
```

▲ 可选的第三方依赖：装不装都能跑

场景	选择	理由
核心流程失败（存库、读配置）	记日志 + 上报用户	静默失败会让数据悄悄丢失
可选依赖缺失	静默降级	功能变弱，但程序可用
清理路径（关连接、恢复界面）	静默忽略	此刻再抛异常只会盖住真正的错
定时器 / 事件回调	记日志 + 保证继续运行	断掉之后不会自己恢复
用户输入格式错	不抛异常，返回 None / 默认值	属于正常业务分支

静默降级最大的风险是「假装没事」

`except: pass` 会让一个功能长期不工作而没人发现。本项目因此定了一条：静默降级必须写明「降级之后是什么状态」。上面那段 `pygments` 的例子，`return None` 的语义是「没有高亮器」，调用方拿到 `None` 会走纯文本渲染——这就是一句能读懂的空兜底，而不是「不知道发生了什么，反正没崩」。

8.5 assert：只用来钉「不可能发生」

assert 会在 -O 优化模式下被整体删掉

`python -O script.py` 时所有 `assert` 语句消失。所以：
绝不能用 `assert` 做输入校验或安全检查（`assert user.is_admin` 这种写法在 `-O` 下直接失效）。
该用的是「内部不变量」——描述「如果这行不成立，说明代码本身写错了」。

```
1 # 内部不变量：锚点必须命中一次，否则说明源码已经被改动过
2 assert raw.count(old) == 1, f"锚点命中 {raw.count(old)} 次，本应恰好 1 次"
```

▲ 本项目所有字节级补丁脚本的第一道闸

项目里 `assert` 出现最多的地方是两个：补丁脚本（保证替换唯一）与回归测试（断言行为）。业务代码里几乎见不到——业务条件一律用 `if` 加正常控制流处理。

8.6 自定义异常

```
1 class CaptureFailed(RuntimeError):
2     """抓图失败（窗口句柄不对、窗口已销毁、或抓到的是纯色空图）。"""
```

▲ scripts/window_shot.py · 一个自定义异常的完整定义只要三行

继承哪个基类是有讲究的：这里选 `RuntimeError` 而不是 `Exception`，含义是「运行环境的问题，不是调用方传错了参数」——后者该继承 `ValueError`，参数类型问题该继承 `TypeError`。选对了，调用方就能写出有意义的 `except`。

```
1 colors = image.getcolors(maxcolors=1 << 20) or []
2 if len(colors) < min_colors:
3     raise CaptureFailed(
4         f"只抓到 {len(colors)} 种颜色（{colors[:1]}）—— PrintWindow 没生效")
```

▲ 抛的时候带上现场数据，收到的人不用再复现一次

自定义异常的规矩

- ① **定义要短**，docstring 说清「什么情况下会抛」；
- ② **基类选对**（`ValueError` / `TypeError` / `RuntimeError` / `OSError`），让调用方能按语义捕获；
- ③ **消息里带上下文**（期望值、实际值、关键变量），不要只写「失败了」；
- ④ 不要为了「显得专业」而给每个函数配一个异常类——本项目的 83 个文件里只定义了一个。

第九章 模块、包与导入

一个 `.py` 文件就是一个模块，一个含 `__init__.py` 的目录就是一个包。本项目的导入语句里藏着几条实用经验，这一章一起讲。

9.1 导入的四种写法

写法	含义	项目里的例子
<code>import sqlite3</code>	导入模块，用时 <code>sqlite3.Row</code>	标准库一律这么导
<code>from pathlib import Path</code>	导入名字	文件路径操作
<code>from datetime import date, datetime, timedelta</code>	导入多个名字	时间计算
<code>import numpy as np</code>	起别名	本项目没用到，但 <code>import tkinter as tk</code> 同理

```
1 from __future__ import annotations
2
3 import calendar
4 import json
5 import sqlite3
6 from dataclasses import dataclass, field
7 from datetime import date, datetime, timedelta
8 from pathlib import Path
9 from typing import Optional
```

▲ `todo_db.py` 的导入区 —— 顺序是「标准库 → 第三方 → 本项目」

导入顺序不是形式主义

PEP 8 建议分三段（标准库 / 第三方 / 本项目），段间空一行。好处很实际：**一眼看出这个模块依赖了几个外部包。**本项目的第三方依赖只有 `openpyxl` 与可选的 `pygments`，所以「第三方段」经常是空的 —— 这本身就是一条信息：**这个程序装起来几乎没有外部依赖。**
`from __future__ import annotations` 必须放最前，它是编译器指令，位置错了会直接 `SyntaxError`。

别用 `from module import *`

```
1 # 不推荐：不知道导进来了什么，还可能覆盖同名变量
2 from todo_db import *
3
4
5 # 推荐：明确列出
6 from todo_db import TodoDB, TodoItem, ALERT_GRACE_MINUTES
```

▲ 对比（项目没有使用星号导入）

9.2 if `__name__` == `"__main__"`

`__name__` 是模块的「名字」：被导入时是模块名，被直接运行时是字符串 `"__main__"`。所以这个守卫的含义是「只有当我被当作脚本运行时才执行」。

```

1  if __name__ == "__main__":
2      sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
3      import tkinter as tk
4
5      root = tk.Tk()
6      root.title("截图自检")
7      root.geometry("320x140+80+80")
8      tk.Label(root, text="PrintWindow 自检", font=("Microsoft YaHei UI", 14)).pack(pady=30)
9      root.update()
10     try:
11         image = shot_widget(root, out)
12         print(f"OK 抓到 {image.size[0]}x{image.size[1]} → {out}")
13     finally:
14         root.destroy()

```

▲ `scripts/window_shot.py` 末尾：既是库，也是可自检的命令行工具

本项目的 `scripts/` 下几乎每个文件都有这一段，好处是同一个文件既能被测试导入，也能单独跑一次做自检——`python scripts/window_shot.py` 会立刻告诉你截图功能在这台机器上可不可用。

守卫里做的事要「无副作用」或明说副作用

`main.py` 是唯一一个没有 `if __name__ == "__main__"` 守卫的主模块吗？并不是——它也有，而且里面只做一件事：建单实例互斥、配日志、起 `mainloop()`。千万不要在模块顶层写有副作用的代码（建文件、连数据库、弹窗），否则别人只是 `import` 一下就触发了。

9.3 包、相对导入与 `__init__.py`

```

1  embedded_admin_tools/
2  |—— __init__.py
3  |—— api_demo_window.py      界面
4  |—— crypto_window.py
5  |—— returnCode.json        数据
6  |—— services/              业务与数据
7      |—— api_client.py
8      |—— crypto_service.py
9      |—— error_code_service.py
10     |—— icon_service.py
11     |—— interface_service.py

```

▲ 内置工具包的分层

```
1 import json
2 import tkinter as tk
3
4 from .services.api_client import ApiClient
5 from .services.crypto_service import DESCryptoService
6 from .services.error_code_service import ErrorCodeService
7 from .services.icon_service import save_temp_icon
```

▲ `embedded_admin_tools/market_quote_window.py` · 用相对导入引同包内的模块

写法	含义
<code>from . import x</code>	同包内
<code>from .sibling import x</code>	同包的子模块
<code>from .. import x</code>	上一级包
<code>from .services.x import Y</code>	同包的子包

`__init__.py` 现在可以省略，但本项目仍然放着

Python 3.3+ 支持「命名空间包」，没有 `__init__.py` 也能导入。但显式放着它有两个好处：① **让工具（打包器、IDE）明确知道这是包**；② 它是「包级初始化」的落点。本项目选择保持显式。

另一个值得记的细节：`embedded_admin_tools/` 根目录下有若干同名文件（如 `error_code_service.py`），它们是带 **docstring** 的 6 行兼容包装，内部转发到 `services/` 里的真实现——为的是老的 `import` 路径不断。看起来像重复代码，其实是有意兼容层，不要删。

9.4 延迟导入：把 800 毫秒挪出启动路径

```
1 def load_excel_assets(file_path: Path) -> list[dict]:
2     try:
3         from openpyxl import load_workbook      # 放在函数里，不放在文件顶部
4     except ImportError as exc:
5         raise RuntimeError("缺少 openpyxl，请先执行 pip install -r requirements.txt") from exc
```

▲ `main.py` · `openpyxl` 按需导入

为什么不用模块级 import

`openpyxl` 的导入要花大约 **800 毫秒**—— 对一个双击就期待看到界面的桌面程序来说，这是能感觉到的延迟。而「导入 Excel」是低频操作，很可能一次都用不上。

判断标准：贵 + 少用 = 延迟导入；便宜 + 常用 = 放文件顶部。延迟导入的代价是「每次调用都要查一次 `sys.modules`」（微秒级，可忽略），以及错误发现得晚（所以要配一个清楚的 `ImportError` 提示）。

延迟导入不能用在「热路径」里

如果一个函数每秒被调用几百次（比如渲染循环、事件处理），在里面写 `from x import y` 虽然 Python 有缓存不会重新加载，但每次仍要走一遍查找与加锁逻辑。**贵的东西放函数里，热的路径别放。**

9.5 冻结（打包）后的路径

```
1 def get_base_dir() -> Path:
2     if getattr(sys, "frozen", False):
3         return Path(sys.executable).resolve().parent
4     return Path(__file__).resolve().parent
5
6
7 BASE_DIR = get_base_dir()
8 DATA_DIR = BASE_DIR / "ExpiryManager_Data"
```

▲ main.py · 一个函数解决「源码运行」与「打包运行」的路径差异

	源码运行	PyInstaller 打包后
<code>__file__</code>	.py 文件的路径	指向临时解包目录（且每次运行都不同）
<code>sys.executable</code>	python.exe 的路径	exe 自身的路径
<code>sys.frozen</code>	不存在	<code>True</code>
<code>sys._MEIPASS</code>	不存在	临时解包目录（放只读资源）

这是打包程序最容易踩的坑

把数据目录建在 `__file__` 旁边，在打包版里会建到临时解包目录——**程序关掉数据就没了**，而且每次运行都从空白开始。正确做法就是上面这三行：**判断 `sys.frozen`，用 `sys.executable` 定位 exe，把数据放在 exe 同级目录**。顺带一个排查技巧：`sys._MEIPASS` 存在就说明正在打包环境里跑，打印它可以确认资源被解包到哪了。

9.6 sys.path 与脚本运行

```
1 if __name__ == "__main__":
2     sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
3     import tkinter as tk
```

▲ scripts/window_shot.py：把项目根目录加进搜索路径，才能 import 项目模块

运行的脚本所在目录会自动进 `sys.path`，但**脚本的父目录不会**。所以scripts/ 下的工具要导入根目录的 `todo_db`，就得自己插一条路径（`parent.parent` 就是根目录）。

sys.path.insert(0, ...) 里的 0

0 表示插到**最前面**，优先级最高。用 `append` 是插到最后，可能被同名的已安装包抢先。这里要的是「优先用项目里的这一份」，所以是 0。

第十章 类型注解

类型注解是给「读代码的人」和「静态检查工具」看的。**运行时它默认不生效**——写成 `def f(x: int)` 之后传字符串照样能跑。理解这一点，就不会把它当成语言层面的类型约束了。

10.1 注解写在三个地方

```
1 count: int = 0 # 1. 变量
2 cache: dict = {} # 有注解就一定会成为 dataclass 字段
3 # (所以类体里给变量加注解要谨慎)
4
5 def snooze(self, item_id: int, # 2. 参数
6     minutes: int = SNOOZE_MINUTES,
7     now: Optional[datetime] = None) -> str: # 3. 返回值
8     ...
```

▲ 三种位置的注解（`todo_db.py` · `snooze` 的签名）

类体里的注解有额外含义

在 `@dataclass` 装饰的类里，**每一条带注解的类属性都会变成一个字段**。所以下面这两种写法语义完全不同：
`x: int = 0` → `dataclass` 认它是字段，进 `__init__` 参数表。
`x = 0` → 只是普通类属性，**不进** `__init__`。
想加一个「不属于构造参数」的类级常量，就不要写注解，或者用 `field(init=False)`。

10.2 Optional 与联合类型

写法	含义	备注
<code>Optional[datetime]</code>	<code>datetime</code> 或 <code>None</code>	等价于 <code>Union[datetime, None]</code>
<code>Union[int, str]</code>	两者之一	3.10 起可写 <code>int str</code>
<code>int None</code>	同上 (3.10+)	旧版本解释器会直接 SyntaxError
<code>Any</code>	任意类型	等于放弃检查，少用
<code>Literal["a", "b"]</code>	限定几个字面量值	很适合表示「状态字符串」
<code>Final</code>	常量标记	<code>Final[int] = 10</code>

```
1 from typing import Optional
2
3 def alert_moment(item: "TodoItem") -> Optional[datetime]:
4     """一条待办「下一次该响」的时刻；没设日期或时间时返回 None。"""
5     if not _norm(item.due_date) or not _norm(item.due_time):
6         return None
7     ...
```

▲ `todo_db.py` · `Optional[...]` 是项目里最常用的类型包装

本项目写 `Optional[X]` 而不是 `X | None`

两种写法在 3.10+ 等价，而且加了 `from __future__ import annotations` 之后连 3.7 都能用 `|` 写法（因为注解不再被求值）。本项目仍统一用 `Optional[X]`，原因是**它也是给不熟悉新语法的读者看的**，而且项目里有一部分模块没有加 `future` 导入。**在同一个代码库里保持一致，比「用上最新语法」更重要。**

10.3 容器泛型

```
1 REPEAT_RULES: list[tuple[str, str]] = [("none", "永不"), ...]
2 LIST_ICONS: list[str] = ["list", "check", ...]
3 moments: dict[int, str] = {}
4 counts: dict[str, int] = {}
5 self._holidays_cache: Optional[dict[str, Holiday]] = None
```

▲ `todo_db.py` · 泛型容器的几种常见嵌套

写法	含义
<code>list[str]</code>	字符串列表
<code>tuple[str, str]</code>	固定两个元素的元组
<code>dict[str, int]</code>	键是 <code>str</code> 、值是 <code>int</code>
<code>list[tuple[str, str]]</code>	二元组构成的列表（本项目的表数据都是这个）
<code>set[int]</code>	整数集合
<code>Sequence[str]</code>	只读序列（比 <code>list</code> 更宽松，参数建议用它）
<code>Iterable[str]</code>	可迭代对象
<code>Iterator[str]</code>	迭代器

注解里的泛型不参与运行

`list[str]` 在运行期并不检查元素类型，它是「给人看的契约」。真要在运行期校验得自己写断言，或者用 `pydantic` 之类的库。本项目选择**不校验**——数据来源是自己写的 SQL 与自己的界面，注解的作用是「读代码时不用猜」。

10.4 Callable：把函数当参数传时怎么标注

```
1 from typing import Callable, Optional
2
3
4 class ConsolePage(ttk.Frame):
5     def __init__(self, master, log_path: Path,
6                   on_status: Optional[Callable[[str], None]] = None):
7         ...
```

▲ console_page.py · 回调参数的标注方式

写法	含义
<code>Callable[[str], None]</code>	收一个 str、返回 None 的函数
<code>Callable[..., None]</code>	参数随便、返回 None
<code>Callable[], None]</code>	无参数、无返回值 (<code>command=</code> 最常见)
<code>Optional[Callable[[int], None]]</code>	可为 None 的回调 —— 传不传都行

`Optional[Callable[...]]` 是界面代码里的标准形状

组件的回调参数几乎总是「可选」的 —— 谁用谁传。项目里所有窗口类的构造参数都遵循这个形状：
`on_changed= / on_open= / on_close=`，默认 `None`，用时先判 `if self.on_open:` 再调用。

```
1         self._on_open = on_open
2         ...
3     def _open(self, item):
4         if self._on_open is None:
5             return
6         self._on_open(item.id)
```

▲ 回调的调用点必须判空（本项目所有组件都遵守）

10.5 from __future__ import annotations

```
1 from __future__ import annotations      # 必须放在文件第一条语句之后
2
3 class TreeNode:
4     def attach(self, parent: TreeNode) -> TreeNode:    # 不用写引号了
5         ...
```

▲ 加了它之后，注解不必是「运行时可求值」的

	不加 future	加了 future
注解求值时机	定义时立即求值	全部变成字符串，永不求值
前向引用	必须写 <code>-> "TodoList"</code>	直接写 <code>-> TodoList</code>
用新语法 (<code>int str</code>)	要 3.10+	3.7+ 都行

	不加 future	加了 future
性能	每个函数定义都要求值注解	零开销
用 <code>get_type_hints()</code>	直接可用	仍可用（会主动求值字符串）

```
1 @dataclass
2 class TodoList:
3     @classmethod
4     def from_row(cls, row: sqlite3.Row) -> "TodoList":    # 没有 future 导入时的写法
5         return cls(...)
```

▲ `todo_db.py` 用了引号包住类名 —— 因为它有 `future` 导入之前的历史写法

引号包类型名是「没有 `future` 导入」时的必需动作

-> `TodoList` 写在 `class TodoList` 内部时，类本身还没定义完，立即求值会抛 `NameError`。加引号让它延迟成字符串即可。**加了 `from __future__ import annotations` 之后就不需要引号了** —— 本项目两种写法并存（老模块带引号、新模块带 `future`），新写代码建议统一带 `future`。

10.6 注解不参与运行 —— 一条必须记住的边界

```
1 >>> def f(x: int) -> str:
2     ...     return x          # 返回的是 int，不是 str
3 >>> f("我甚至传了个字符串")
4 '我甚至传了个字符串'        # 一切正常，没有任何报错
```

▲ 注解完全不影响运行

想要运行期检查，有三条路：**手动 `isinstance`、用 `pydantic` / `attrs` 这类校验库、或者靠测试**。本项目走第三条 —— 1053 个断言构成的回归测试就是它的「类型检查器」，而且是**检查行为而不是检查声明**的那种。

类型注解在本项目的实际定位

它不是约束，而是**文档**。价值体现在三处：① IDE 的自动补全与跳转；② 读代码时不用翻实现就能知道参数形状；③ `@dataclass` 靠它生成 `__init__`（这是唯一「有实际作用」的用法）。

所以项目里**没注解的地方也不必补** —— 补了不影响运行，漏了不出错。但 `@dataclass` 的字段必须有注解，那是硬要求。